

LEVEL 8 — Statistical Functions in NumPy

Beginner-friendly, line-by-line examples with output for Data Analyst interview preparation.

1. Setup

```
Line 1: import numpy as np
Line 2: data = np.array([10, 20, 30, 40, 50])
Line 3: print(data)
```

Output: [10 20 30 40 50]

2. np.mean()

Finds the arithmetic average.

```
data = np.array([10, 20, 30, 40, 50])
result = np.mean(data)
print(result)
```

Output: 30.0

3. np.median()

Finds the middle value after sorting the data.

```
data = np.array([10, 20, 30, 40, 50])
result = np.median(data)
print(result)
```

Output: 30.0

4. np.average()

Calculates an average. It can also calculate a weighted average.

```
data = np.array([10, 20, 30, 40, 50])
result = np.average(data)
print(result)
```

Output: 30.0

5. np.average() with weights

Useful when some values have more importance than others.

```
marks = np.array([60, 70, 80])
weights = np.array([1, 2, 3])
result = np.average(marks, weights=weights)
print(result)
```

Output: 73.33333333333333

6. np.std()

Finds standard deviation, which measures how spread out values are.

```
data = np.array([10, 20, 30, 40, 50])
result = np.std(data)
print(result)
```

Output: 14.142135623730951

7. np.var()

Finds variance, another measure of spread.

```
data = np.array([10, 20, 30, 40, 50])
result = np.var(data)
print(result)
```

Output: 200.0

8. np.min()

Finds the smallest value.

```
data = np.array([10, 20, 30, 40, 50])
result = np.min(data)
print(result)
```

Output: 10

9. np.max()

Finds the largest value.

```
data = np.array([10, 20, 30, 40, 50])
result = np.max(data)
print(result)
```

Output: 50

10. np.ptp()

Finds the range: maximum value minus minimum value.

```
data = np.array([10, 20, 30, 40, 50])
result = np.ptp(data)
print(result)
```

Output: 40

11. np.percentile()

Finds a value below which a given percentage of observations falls.

```
data = np.array([10, 20, 30, 40, 50])
result = np.percentile(data, 50)
print(result)
```

Output: 30.0

12. np.quantile()

Similar to percentile, but the input is written as a fraction from 0 to 1.

```
data = np.array([10, 20, 30, 40, 50])
result = np.quantile(data, 0.5)
print(result)
```

Output: 30.0

13. np.argmax()

Returns the index position of the largest value.

```
data = np.array([10, 50, 20, 40, 30])
result = np.argmax(data)
print(result)
```

Output: 1

14. np.argmin()

Returns the index position of the smallest value.

```
data = np.array([10, 50, 20, 40, 30])
result = np.argmin(data)
print(result)
```

Output: 0

15. Statistics by Axis

For a 2D array, axis=0 works down the rows and gives one result for each column. axis=1 works across the columns and gives one result for each row.

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

```
print(arr)
```

Output:

```
[[10 20 30]
 [40 50 60]]
```

16. Mean by row — axis=1

```
result = np.mean(arr, axis=1)
print(result)
```

Output: [20. 50.]

17. Mean by column — axis=0

```
result = np.mean(arr, axis=0)
print(result)
```

Output: [25. 35. 45.]

18. Sum by row — axis=1

```
result = np.sum(arr, axis=1)
print(result)
```

Output: [60 150]

19. Sum by column — axis=0

```
result = np.sum(arr, axis=0)
print(result)
```

Output: [50 70 90]

20. Standard deviation by row — axis=1

```
result = np.std(arr, axis=1)
print(result)
```

Output: [8.16496581 8.16496581]

21. Standard deviation by column — axis=0

```
result = np.std(arr, axis=0)
print(result)
```

Output: [15. 15. 15.]

22. Variance by row — axis=1

```
result = np.var(arr, axis=1)
print(result)
```

Output: [66.66666667 66.66666667]

23. Variance by column — axis=0

```
result = np.var(arr, axis=0)
print(result)
```

Output: [225. 225. 225.]

24. Interview Quick Revision

Function	Purpose
np.mean()	Average
np.median()	Middle value
np.average()	Average / weighted average
np.std()	Standard deviation
np.var()	Variance
np.min()	Smallest value
np.max()	Largest value
np.ptp()	Maximum – minimum
np.percentile()	Percentile value
np.quantile()	Quantile value
np.argmax()	Index of maximum
np.argmin()	Index of minimum
axis=0	Calculate down rows → one result per column
axis=1	Calculate across columns → one result per row

Important interview tip: Remember axis=0 as column-wise calculation and axis=1 as row-wise calculation for a 2D NumPy array.